

Division of Computing Science and Mathematics
Faculty of Natural Sciences
University of Stirling

MATPMD4 - Stochastic Processes and Optimisation

Assignment 2 - Optimisation

Student Number 3541011

April 2026

Contents

Part One – Maximising a function	1
Part Two – Distribution Network	1
Part Three – Own real-life example	2
1. Background	2
2. Aim.....	2
3. Model	2
4. Optimisation Methods	3
5. Results and Optimality	4
6. Conclusion and Critical Analysis	6
Reference List.....	7
Appendix	8
1 – Dataset Table	8
2 – Manual Drawing – Routes taken by the vehicles	9

Part One – Maximising a function

The first task of this assignment is to maximise the given function $f(x,y,z)$ with the following constraints: $0 \leq x,y,z, \leq 5$.

I started by running the algorithm 20 times with starting points at random. Looking at the results, some runs produced modest outcomes like $(x,y,z) = 2.04, 2.50, 3.21$ with $f = 5.95$; on the other hand, other runs performed much better hitting scores of $(x,y,z) = 0.98, 2.07, 2.15$ with $f = 7.36$ or even better $(3.84, 0.78, 0.02)$ with $f = 7.69$.

Analysing the results, the best output is $(x,y,z) = 4.57, 1.21, 0.55$ with $f = 7.96$.

As an extra step, I ran again the hill climb but starting from the best point using x, y and z from above, instead of starting at random. More specifically, I started the runs at $(x,y,z) = (4.569, 1.205, 0.554)$ with $f = 7.956$ roughly.

The output was very similar with only a few runs performing better, such as $(4.570, 1.205, 0.556)$ with $f = 7.958$ or $(4.570, 1.206, 0.554)$ with $f = 7.964$. There were no values across the 20 runs analysed that discovered a value considerably larger than my best result. To conclude, this method hit the best results around the region of $(4.57, 1.21, 0.55)$ with $f = 7.96$.

Part Two – Distribution Network

Given the aim is to minimise the total daily costs, find the best strategy such that every store receives its delivery, and the warehouses have the correct number of vehicles at the end of the day to carry out the deliveries the following day.

Questions: Which stores should each warehouse supply? How many vans or lorries does each warehouse require? What routes should each vehicle take? What is the total cost?

Logistic Route:

W1 – First Van – Green – W1 – 1 – 12 – 19 – 11 – 6 – W1 (121.23 miles)

W1 – Second Van – Red – W1 – 20 – 14 – 17 – 8 – W1 (87.55 miles)

W2 – First Van – Yellow – W2 – 5 – 2 – 4 – 10 – 22 – W2 (152.35 miles)

W2 – Second Van – Orange – W2 – 7 – 9 – 13 – 15 – 18 – W2 (104.43 miles)

W2 – Third Van – Grey – W2 – 23 – 21 – 3 – 16 – W2 (92.96 miles)

The route above covers the 23 stores using two vans for the first warehouse (covering 10 shops) and three vans for the second warehouse (delivering to 13 stores). The routes that I have chosen ensures that the vehicles can visit four to five stores per trip. See manual drawing in Appendix 2.

The miles involved are $121.23 + 87.55 + 152.35 + 104.43 + 92.96$ totaling up to 558.52 miles. As I am only using vans, the total cost per day is therefore $558.52 \times 2 = £1117.04$. Arguably, given the fact that a lorry would cost me £3 per mile, the exact same route would be more expensive by x1.5 compared to my current option.

The counterargument of using a lorry and covering up to 18 stores would mean that the driver would need to cover a wider region in one drive, leaving 5 stores out due to capacity and running at £3 per mile. It is more cost-effective to employ smaller vans for the runs instead of one van and one lorry. In that case the lorry would have a surplus in price of roughly £800 depending on the store visited and route given that it would cover roughly 300 miles. In my current case, the five vans employed have a cost of £1117ca and the mean mile is roughly 110 equaling to £220ca. The important element is that the cost is per mile and not per vehicle so logistically it makes sense to have shorter runs. Critically, if the example had cost per car and the stores were positioned in cluster angles and only W1 would exist, then the best option would be to be to use as few lorries as possible that would deliver to the nearby stores instead of multiple vans like in my case.

Part Three – Own real-life example

1. Background

This example relates to an issue in my real life given my passion for drawing and art. The problem that I am looking to solve is as follows: with £100 maximum expenditure, what art supplies should I buy for personal use? To start this project, I took the data (cost of each product) from various websites that I regularly use and assigned it a subjective score that focuses on how useful it is to me. See Appendix 1 for data on a clear table. Given the budget set, not all products can be bought, which was a decision I took to mark the tradeoff between how useful the item would be and its cost.

2. Aim

The aim of this project is to maximise the usefulness of the item I can buy, but it must be within the budget cap set of £100.00.

The objective is to find the best score of usefulness but to stay within the set budget.

The contents I identified are as follows:

- Must be within budget
- Every product can either be purchased or not purchased, so fractional items are not accepted.

3. Model

I have modelled the problem as 0 and 1 knapsack:

I included 15 products ($n = 15$) and for each one of the items i :

- *Weights* $[i] = [5, 8, 6, 3, 7, 15, 6, 18, 20, 6, 10, 12, 4, 7, 15]$, which are the approximate prices of the items in GBP.
- *Values* $[i] = [9, 7, 8, 6, 8, 9, 7, 9, 8, 7, 8, 8, 5, 6, 7]$, which refer to how useful the items are.

To note is the binary vector sol of length of n with $sol[i]$ refers to purchase items while 0 refers to will not purchase it. The total cost of a solution would be the total cost of the prices of all the products I am interested in buying but with my £100 budget. In addition, the total value of a

solution is the sum of the usefulness scores of all the items I can pick and that's exactly what the optimisation methods below aim to maximise within budget.

In essence, the aim is to maximise *value (sol)* keeping in mind the *cost (sol) ≤ 100* and *sol[i] ∈ {0, 1}* for every item.

4. Optimisation Methods

In the following project, I have adopted three methods:

- Random search
- Hill Climb
- Dynamic Programming

Random Search

I computed *steps_RS = 1000* iterations, and the algorithm created a random vector 0 and 1. It analyses the item one by one and with a probability of 0.5 per item, it sets *sol[i]* to 1. Though, it reverses and turns back to 0 if it goes over the £100 limit. In addition, every time it generates a solution, it adds the total usefulness score and records the best solution till that moment.

Hill Climb

It starts by setting a random feasible solution by randomly assigning 0 and 1 and turning down any solutions over £100. Then, with *steps_HC = 1000000* (one million) iterations, it replicates the best solutions and flips one random chosen (1 to 0 or 0 to 1). If it goes over, it is not taken into consideration: alternatively, if it is better, it gets replaced and with the higher value.

Dynamic Programming

I built a table with *dp_table[i][w]* where each entry is the best total usefulness I can get using the first *i* products with a budget *w* from 0 to 100. Then, *i* the table is filled one row at a time and work backwards from *dp_table[n][100]* to see which products were truly selected as the best shopping list for me. To ensure accuracy, I used the same weights, values, and cap for all three codes.

5. Results and Optimality

Dynamic Programming

This process follows the methods described by Kellerer, Pferschy and Pisinger where as they argue that the DP table and the algorithm explores all items analysed and produce the best possible answer [1]. More specifically, the total usefulness of 91 with £100 budget, picking the following items: [1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13]. I am confident that this is the best solution, given that the DP table takes into account all possible capacities up to 100 for the products analysed. This mirrors Kellerer, Pferschy and Pisinger's arguments, who note that dynamic programming is a global optimum bundle and it can be a powerful tool as an exact benchmark with hill climbing and random search [1].

Random Search

The best solution identified with this search was 88 with a cost of £97 with final items: [1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1]. This outcome is slightly lower than dynamic programming where the optimum is 91. This proves that RS is able to find high value bundles but clearly is not capable of always reaching the global optima within the selected numbers of steps. This idea is highlighted by Bergstra and Bengio as they argue that the quality of the outcome will depend on the number of random trials [2]. To specify, the best outcome produced is 88, which is close to the DP optima but still not the best.

Hill Climb

The worst performance was given by the hill climb with the best value found at 84 with the cost of £100 for items as follows: [1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1]. Given the random start at a feasible point and the flipping mechanism one by one, it does improve over some periods, but it gets stuck in a local optimum below the threshold of 91 showed in dynamic programming. On the same note, Samir et al. argue that even if this method can be fast and gives a good enough answer, it does not guarantee that a global optimum can be identified [3]. To add, they further explain that this search only explores neighboring configurations instead of all possible locations. Looking at my analysis, even with a million iterations, the flip hill improves step by step over random searches, but it remains stuck on sub-optimal hills. Critically, this shows the tradeoff between guaranteed optimality and speed as well as efficiency.

Optimal and closeness to optimal

As the dynamic programming takes into consideration all combinations possible up to its capacity, it can be stated that the global optimum for this model is 91. The random search hit 88 and the hill climb only achieved 84, which are very close but still not the best value. Critically, it proves that:

- A global maximum exists (at least one) of 91 with the set given.
- Hill climb and random search do indeed get close to the best optimum, but they do struggle when having the constraint of the number of iterations and budget. In fact, this suggests local optima.

6. Conclusion and Critical Analysis

To sum up, this project analysed how a shopping decision from my life can be turned into an optimisation process and tackled by using various algorithms.

Strengths:

- Dynamic programming illustrated the exact benchmark, so it is very clear how the other two methods (hill climb and random search) position themselves across the three searches.
- By also using hill climb and random search, I have clearly demonstrated that these two methods can improve solutions but can also pose a risk when looking to identify global optimum.

Weaknesses and Limitations:

- I made up the usefulness scores based on my general idea. In reality, I would pay closer attention to what type of paper, brush, and colour I buy based on the piece of art I am working on. The model does not know this, and it cannot be considered a true reality all the time.
- In the real world, cashbacks, bundle offers, and multi-packs are very common when buying such products. The model does not know this and does not consider sales or any promotions that I could find online, which would always maximise my purchasing power.

To conclude, this assignment showcased how several optimisations' models acted on a realistic problem in my life. Dynamic programming was able to hit the true optimum, while hill climb and random search hit a near-optimal outcome but also supported critical thinking regarding global and local optima.

Reference List

- [1] H. Kellerer, U. Pferschy, and D. Pisinger, “Exact solution of the knapsack problem,” in *Knapsack Problems*, Berlin, Germany: Springer, 2004, pp. 117–160.
- [2] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, no. 2, pp. 281–305, 2012.
- [3] B. A. L. B. A. L. Samir, L. Yacine, and B. Mohamed, “Local search heuristic for multiple knapsack problem,” *International Journal of Intelligent Information Systems*, vol. 4, no. 2, pp. 35–39, 2015
- [4] Tate, “Tate Shop – Art materials and stationery,” Tate, London. Available: <https://shop.tate.org.uk>. Accessed: Apr. 2, 2026.
- [5] Royal Academy of Arts, “Art Materials – Royal Academy Shop,” Royal Academy of Arts, London. Available: <https://shop.royalacademy.org.uk>. Accessed: Apr. 2, 2026.

Appendix

1 – Dataset Table

#	Target	Product	Price	Website
1	£5	A6 Postcard	£5	Shop.tate.org.uk
2	£8	Watercolor Paint Set	£8	Shop.tate.org.uk
3	£6	Watercolor Pencil Set	£6	Shop.tate.org.uk
4	£3	Paint Palette	£3.50	Shop.tate.org.uk
5	£7	British Art Activity Book	£7.25	Shop.tate.org.uk
6	£15	Box of 16 washable Felt Pens	£15	Shop.tate.org.uk
7	£6	Navy A6 Royal Academy Watercolor Sketchpad	£6	Shop.royalacademy.org.uk
8	£18	Mini Landscape Painting Kit	£18	Shop.royalacademy.org.uk
9	£20	Neon Coloring Pencil Set	£20	Shop.tate.org.uk
10	£6	Acrylic Paint Brush Set	£6.99	Shop.tate.org.uk
11	£10	Sonography Postcard Kit	£10	Shop.tate.org.uk
12	£12	David Shrigley “My Artwork is Terrible” Sketchbook	£12	Shop.tate.org.uk
13	£4	Sally Muir Mini Notepad	£4	Shop.royalacademy.org.uk
14	£7	Blue A5 Royal Academy Sketchpad	£7.95	Shop.royalacademy.org.uk
15	£15	Seascape Oil Pastels Set of 6	£15	Shop.royalacademy.org.uk

2 – Manual Drawing – Routes taken by the vehicles

